战队名称

奶龙

比赛排名

总分数：4219

学生组/大一新生组排名：1

总排名：14

战队成员

qllz、王思宇

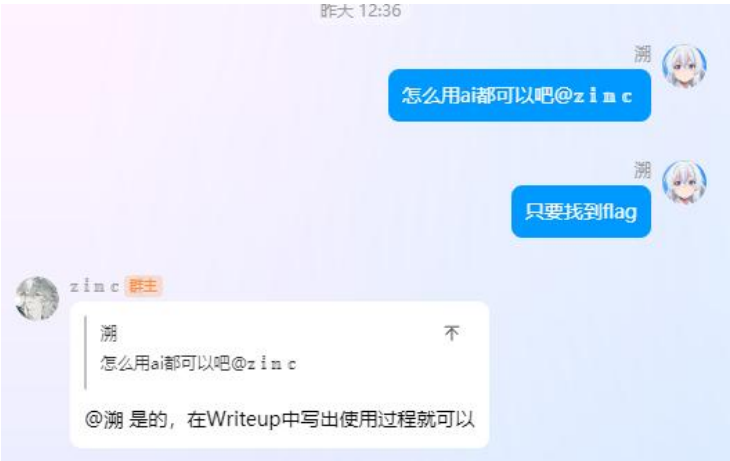
Wa1ting4u、杨俊雷

Source、刘宸伯

签到

上传了学生卡背面的照片

并且得到了 ai 使用许可



easyAES

1、第一步。

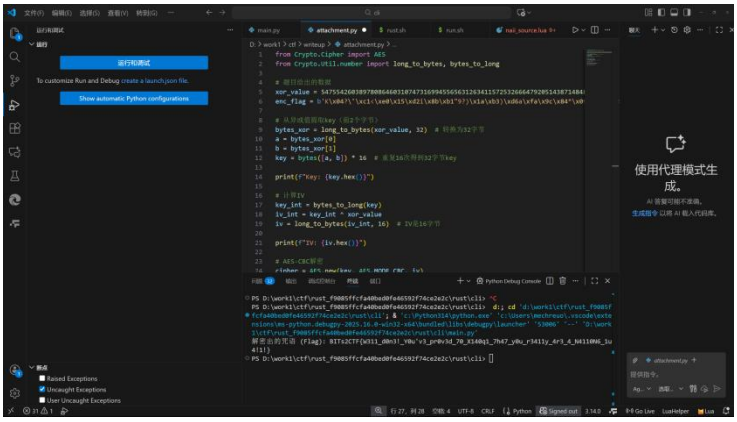
使用 ai

2、第二步。

让 ai 给出代码

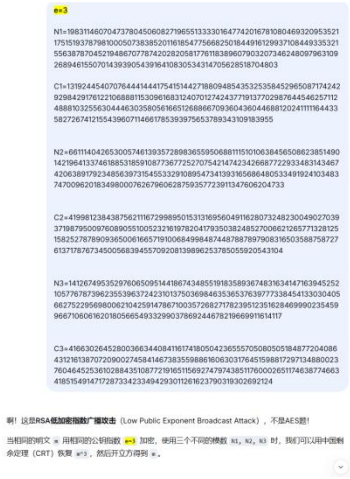
3、第三步。

在 vs 里运行代码，得到 flag 多少，忘了，图里是我刚找的历史记录粘上去的



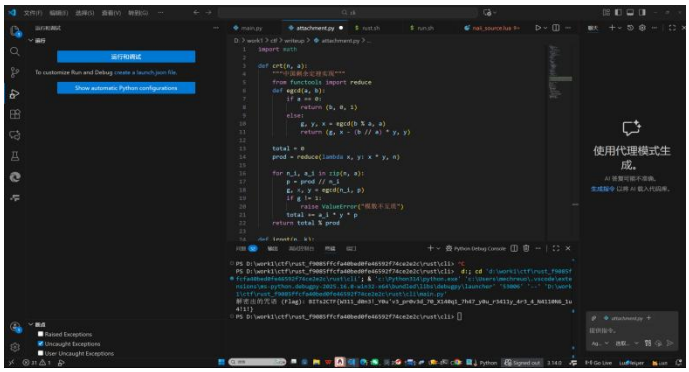
Remaining thing

1. 使用 ai



2. ai 给出代码

3. 运行 ai 给的代码，得到 flag，

3. 

1zvm

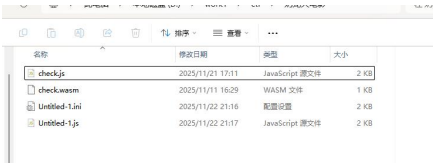
1. 把题目告诉 ai

2. 复制 ai 代码给沙箱

（ai 太好用啦）

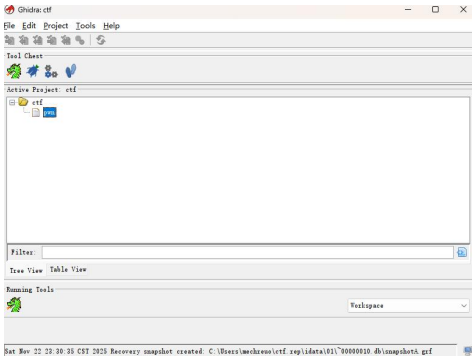
奶龙与小七之大战 webassembly

1. 下载附件

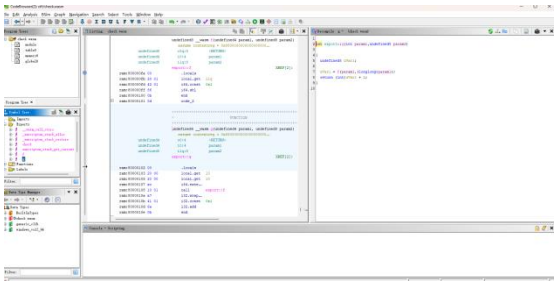


2 把 check 的代码发给 ai

2. 按 ai 说的安装 ghidra



3. 发现没有对应语言，上网找到了 wasm 的反编译插件，安装，反编译



4. 按照 ai 要求把一些函数的伪代码发他（check，g，f）

uint export::check(int param1,uint param2,uint param3)

{
uint uVar1;...

这是您在 Ghidra 或 WasmExplorer 中从 `check.wasm` 文件反编译得到的伪代码，它定义了 Wasm 的核心验证逻辑。

Wasm `check` 函数分析

这段伪代码揭示了 Flag 字节的验证公式。我们将它映射回 JavaScript 中的参数：

Wasm 伪代码参数	JavaScript 参数	含义
param1 (int)	i	索引
param2 (uint)	flag[i].charCodeAt()	Flag 字节
param3 (uint)	enc[i]	加密字节

导出到 Google 表格

1. 验证公式

`check` 函数必须返回 `1` (true) 才能通过验证。函数末尾的表达式就是验证条件：

Decompile: check - (check.wasm)

```
uint export::check(int param1,uint param2,uint param3)

{
  uint uVar1;

  uVar1 = g(param1,(longlong)(param1 << 1));
  return (uint)((uVar1 ^ param2 ^ param3) == 8);
}
```

5. ai 给出解密脚本

特洛伊木马感染典型现象分析

`check` 函数的验证公式是：

$$\text{flag}[i].\text{charCodeAt}() = K_i \oplus \text{enc}[i] \oplus 8 $$代入 K_i：$$\text{flag}[i].\text{charCodeAt}() = (2i + 1) \oplus \text{enc}[i] \oplus 8 $$

最终解密脚本

我们可以使用这段公式编写一个 Node.js 或 Python 脚本来计算 Flag 的每一个字节。

JavaScript

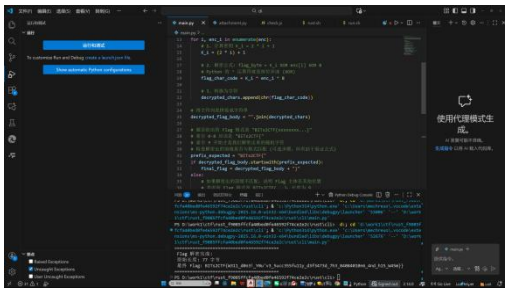
```
/**
* BITs2CTF Wasm 逆向解密脚本 (Node.js)
* 解密公式: flag[i] = ( (2 * i) + 1 ) XOR enc[i] XOR 8
*/

const enc = [
75, 66, 89, 124, 51, 64, 81, 65, 98, 76, 46, 46, 32, 76, 113, 39, 71, 24, 12, 112,
120, 19, 80, 0, 79, 8, 98, 10, 68, 80, 86, 4, 124, 126, 43, 58, 112, 114, 60, 24,
61, 104, 59, 108, 101, 100, 102, 51, 54, 92, 5, 92, 62, 91, 81, 87, 65, 79, 77,
78, 65, 29, 67, 40, 189, 229, 233, 208, 233, 178, 176, 216, 206, 175, 168, 242, 23
];

let decryptedFlag = "";

for (let i = 0; i < enc.length; i++) {
const enc_i = enc[i];
```

6. 运行，得到 flag



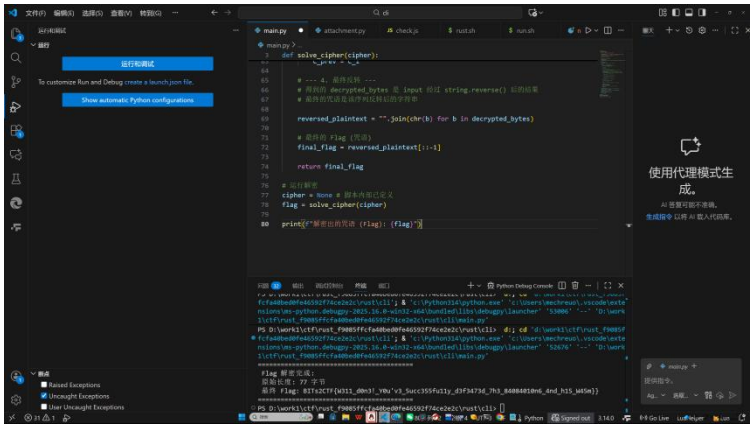
奶龙与小七之真假奶龙

1. 下载附件，观察题目，学习 lua 逆向相关知识，发现附件的.out 文件开头是 lua 文件头

2. 寻找 lua 逆向工具，luadec，逆向失败，因为最高支持 lua5.3，题中文件是 5.4，寻找新的 lua 逆向工具，unluacv，逆向失败，版本老，终于在不知道那找到了支持 5.4 的 unluacv，成功逆向，得到 lua 源码，发给 ai

3. ai 生成脚本

4. 运行，得到 flag



A. 故障机器人

1. 观察题目，发给 ai
2. Ai 给出程序

请输出刚才那题的C++代码

好的，我将为您输出刚才那道“故障机器人”问题的 C++ 实现代码。这个代码将采用我们推导出的 $O(N \log N)$ 算法，并处理大整数（a_i 最高可达 10^9）以及最终结果（总伤害）可能很大（最高可达 10^{14}，需要使用 `long long` 类型）的情况。

故障机器人问题的 C++ 代码

C++

```cpp
#include <iostream>
#include <vector>
#include <algorithm>
#include <cmath>

using namespace std;

// 使用 long long 类型来存储血量和总伤害，以防止溢出。
typedef long long ll;

/**
 * @brief 求解故障机器人问题，计算受到的总伤害。
 * * 核心公式：总伤害 D = Sum_{i=1 to n} [ (n - i + 1) * (R_i - 1) + (n - i) ]
 * 其中 R_i = ceil(a_i / 3)，是杀死第 i 只异鸟所需的回合数。
 */
void solve() {
    int n;
    // 使用 cin/cout 加快输入输出，对于 n=10^5 是必要的
    if (!(cin >> n)) return;
```

3.发到线上 oj，通过，得到 flag

B. QwQwQwQ

与上题相同

好的，我们来编写 **Problem B: QwQwQwQ** 的 C++ 代码。

这个问题的核心是利用递归和倍增思想在指数级增长的字符串中定位字符。由于 k 很大，我们需要一个快速查找函数 `FindChar(k, x)`。

💻 QwQwQwQ 问题的 C++ 代码

C++

```cpp
#include <iostream>
#include <string>
#include <vector>
#include <cmath>
#include <algorithm>

using namespace std;

// 使用 long long 类型处理 k 和 x 的巨大数值 (<= 10^9)
typedef long long ll;

// 预计算 L_k，即对单个 'q' 进行 k 次操作后的长度。
// L_k = 2^(k+1) - 1.
// 我们只需要计算到 L_k > 2 * 10^9 即可，因为 x <= 10^9。
// 预计算的数组大小：k 最大约 30 即可覆盖 10^9 的范围。
const int MAX_K_PRECALC = 35;
ll L[MAX_K_PRECALC];

/**
 * @brief 递归查找对 'q' 进行 k 次操作后，第 x' 个字符是什么。
 * @param k 剩余操作次数
 * @param x_prime 相对位置 (1-based)
 * @return 目标字符
 */
char recurse_find_char(ll k, ll x_prime) {
    // 基础情况：k=0，字符就是 'q'
    if (k == 0) {
```

提交，得到 flag

题目：babyRSA

第一步：使用 AI 分析题目代码：

第二步：验证答案是否正确；

```
def bytes_to_long(b):
    return int.from_bytes(b, byteorder='big')


def long_to_bytes(n):
    l = (n.bit_length() + 7) // 8
    return n.to_bytes(l, 'big')


# 输入题目给的 N,m1,m2
N =
```

636585149594574746909030160182690866222909256464847291783000651837227921337237899651287943597773270944384034858925295744880727101606841413640006527614873110651410155893776548737823152943797884729130149758279127430044739254000426610922834573094957082589539445610828279428814524313491262061930512829074466232633130599104490893572093943832740301809630847541592548921200288222432789208650949937638303429456468889100192613859073752923812454212239908948930178355331390933536771065791817643978763045030833712326162883810638120029378337092938662174119747687899484603628344079493556601422498405360731958162719296160584042671057160241284852522913676264596201906163

m1 =
437018910555377754758713090674828434552623609418189605883878395265943244846522356956125593648928796885738904275195790322403904731577406221662800615649192135096594758567990700774710458621606544804995324319230202175408806666706760877979225732807083632781339065831416759856459922014414321419450650531126833451929640356477652908076978264462932182064696092260867124176080216707230318582489201642918604678896800531886883658762692711484113665211145799415150955794231203475326910050714552990789066306397500097510688331809380587538041735280652368532667434420432748055562621692991174851064017650989088686897296287498777228019985869798956067763811879178829955994366

m2 =
423775156887345771483214073174563566225971899378137973113300330050744806291600501030439300493007373439572486576854900377183900568124875648580405117900334763550734704850545525195312282286018918692626766754698715797675242649987830123254075074849260132465272799050535715612790905925076809808368519625239911812635174499655330946748878170282554491220873730105594218997553839772828068688676446878747560971142890996647615848402287281884093406826017411838889609249863677317852336524497456996075022130956047060027503791571428645322713377816344348903224059331111656623916885513248288451490721944450552506300046606050837515709217964541177043512544147327439793371612

e = 0x10001

flag = b"BITs2CTF{isjna_cask_asdasfasqwe_dmqwo}"

```
# 拆成前后两半
half = len(flag)//2
F1 = bytes_to_long(flag[:half])
F2 = bytes_to_long(flag[half:])

# 计算 S,T
c1 = F1 + F2
c2 = F1**3 + F2**3

# 模幂
def mod_pow(a, b, n):
    return pow(a, b, n)

# 进行 RSA 加密
test_m1 = mod_pow(c1, e, N)
test_m2 = mod_pow(c2, e, N)

print("m1 match?", test_m1 == m1)
print("m2 match?", test_m2 == m2)
```

答案正确 得到 flag: BITs2CTF{isjna_cask_asdasfasqwe_dmqwo}

题目：http

第零关：Chrome 浏览器->右上角三个点->更多工具->开发者工具：

找到：

<!--恭喜你学会查看网页源代码-->

<!--前往/re9uest.php 进入下一关-->

第一关：使用 Postman 发送：

Post:http://127.0.0.1:64481/re9uest.php?b1ts[]=ctf&b1ts[]=f1ag

Body:

Key:byte Value:123

得到：

<h1>欢迎来到第 1 关，这里考察最基本的 http 请求</h1>

使用 GET 发送一个数组 b1ts, 其中依次包含两个值, ctf 和 f1ag

使用 POST 发送一个参数 byte, 其值为 123

恭喜过关，请前往 /ta5k2.php

第二关：访问/robots.txt

User-agent: *

Disallow: /r0b0t.php

User-agent: Yandex

Disallow: *

得到: /r0b0t.php

第三关:

在 Postman 中添加以下 Headers:

X-Forwarded-For:10.194.143.89

Referer: yandex.com

Client-IP:127.0.0.1

X-Real-IP:127.0.0.1

Host:127.0.0.1

Client-IP:127.0.0.1

Forwarded-For-Ip: 127.0.0.1

Forwarded-For: 127.0.0.1

Forwarded-For: localhost

Forwarded:127.0.0.1

Forwarded: localhost

True-Client-IP:127.0.0.1

X-Client-IP: 127.0.0.1

X-Custom-IP-Authorization: 127.0.0.1

X-Forward-For: 127.0.0.1

X-Forward: 127.0.0.1

X-Forward: localhost

X-Forwarded-By:127.0.0.1

X-Forwarded-By: localhost

X-Forwarded-For-Original: 127.0.0.1

X-Forwarded-For-original: localhost

X-Forwarded-For: 127.0.0.1

X-Forwarded-For: localhost

X-Forwarded-Server: 127.0.0.1

X-Forwarded-Server: localhost

X-Forwarded: 127.0.0.1

X-Forwarded: localhost

X-Forwared-Host: 127.0.0.1

X-Forwared-Host: localhost

X-Host: 127.0.0.1

X-Host: localhost

X-HTTP-Host-Override: 127.0.0.1

X-Originating-IP: 127.0.0.1

X-Real-IP: 127.0.0.1

X-Remote-Addr: 127.0.0.1

X-Remote-Addr: localhost

X-Remote-IP: 127.0.0.1

得到：

<h2>本关只允许从本地进行访问！</h2>

访问 /f0ur.php 进入下一关

第四关

在 Cookie 中，设置为：

admin=1; Path=/; Expires=Sun, 23 Nov 2025 04:05:31 GMT;

发送 Post 请求得到：

<h2>请以 admin 的身份访问此页面</h2>

恭喜，你已经掌握基本的 HTTP 知识了

这是你的奖励：
PD9waHAgJ0JJVHMyQ1RGezM4NmZmMTFiLTJiNGQtNDQxZS1hNWZlLTFiMmEzODQ2M2JhYX0nID8+

这是一个 Base64 编码 解码得到：

<?php 'BITs2CTF{386ff11b-2b4d-441e-a5fe-1b2a38463baa}' ?>

即 flag

Are you crazy

Dpsk 秒了

https://chat.deepseek.com/share/ms2duoi12v00xzseo8

输出的程序有点问题，自己修修改改就行了

最终解码程序：

Decrypt.py

Chacha20

题目给出一个 pyc 文件，但由于 py 为解释性语言，pyc 可以被轻松反编译得到源代码，甚至有在线逆向站 python 反编译 - 在线工具

于是我们得到了 chacha20.pyc 的源代码

chacha20.py

再将逆向得到的代码喂给 ai 分析得到加密方法：

1. 生成随机密钥 k1 和随机数 n1

2. 使用当前时间秒数取模 100 作为随机种子

3. 生成随机密钥 k2（128 位）和随机数 n2（64 位）

4. 最终密钥 = k1 ^ k2，最终随机数 = n1 ^ n2

random.seed(int(time.time()) % 100)，这意味着随机种子只有 100 种可能的值

所以只需要用 0-99 代入随机种子暴力碰撞即可

decrypt.py

我把压缩包密码藏在了一个神秘的地方（未解出）

显然，甩给 ARCHPR 爆破一下得到密码 20251122

解密出的俩玩意儿看 010 有一堆视频编码信息（注意到一堆 FF 但何意味），格式换成 MP4，然后看到一坨

然后不会了 :）不是欺负我还没学过工图是吧。。。。。。

Math

拿到手，甩给 exeinfope 看一下，嗯没加壳，那就直接甩给 ida 分析

然后得到源代码的关键逻辑部分

根据其中 judge 和 fun1234 的判断逻辑可以列出六元方程组

(a + b) % 951081 = 597141

(a - b) + 2*c = 1644082

(4*f) XOR d = 1161537

d - e = 68778

a + f = 1136538

2*d + e = 1952901

方程组直接甩给 ai 解出即为答案 678737 869485 917415 673893 605115 457801

Eaze_maze

拿到手，甩给 exeinfope 看一下，嗯没加壳，那就直接甩给 ida 分析

然后得到源代码的关键逻辑部分

maze.exe.i64

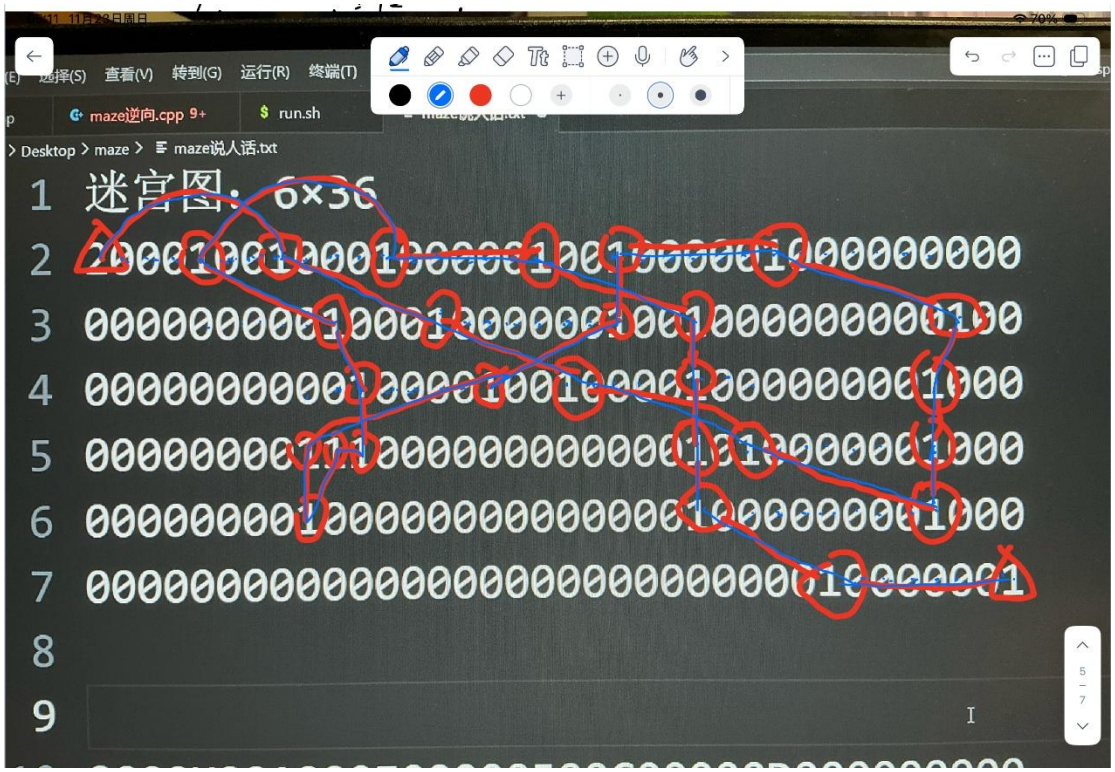
maze.cpp

尝试甩给 ai 分析无果，ai 太蠢把自己绕进去了

自己分析，列出地图

maze说人话.txt

观察到地图和移动规则有显著的特征，似乎可以人肉分析，于是：



人肉爆破迷宫说是

GCC 偷走了重要的函数（未解出）

（总感觉死在了奇奇怪怪的地方）

（这道题感觉好像很久之前在 B 站上哪个视频看到过，是与程序空间极限压缩有关的）

```
main(){puts("Hello World");}
```

这串代码在本地的Linux（Ubuntu22.04）环境下可以过测

但打死也没连上云环境，遂褒姒